Computer Architecture (Practical Class)
Assembly: Stack and Procedure Calls

Luís Nogueira Nuno Pereira Raquel Faria

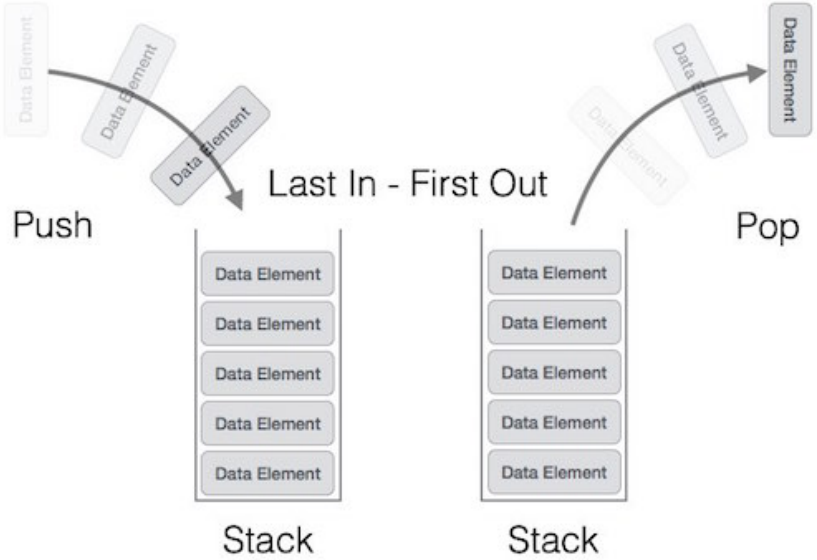
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

{lmn,nap,arf}@isep.ipp.pt

2018/2019

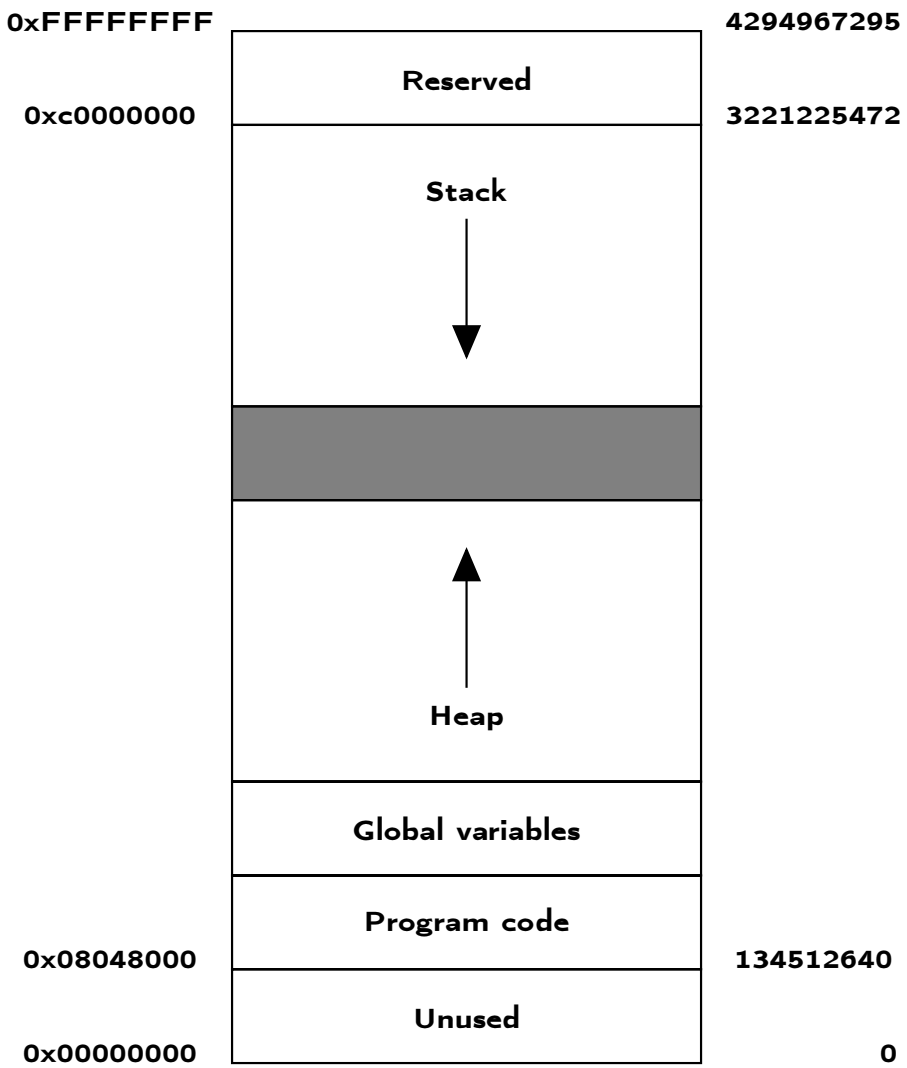
The stack

- A structure of type LIFO (Last In/First Out)
- A new element can only be added to the top, becoming the new top
- Existing elements can only be removed from the top



The stack in IA32

- Region of memory managed with stack discipline
- Grows toward lower addresses
- Register %esp indicates lowest allocated position on the stack (i.e., address of top element)



The PUSH and POP instructions

`pushl source`

- Decrements %esp by 4 bytes
- Writes `source` at address given by %esp

`popl destination`

- Reads the value at address given by %esp and writes it to `destination`
- Increments %esp by 4 bytes

- PUSH and POP accept 16- and 32-bit operands
- Therefore, only the `l` and `w` variants can be used
- The ESP register is updated by 4 or 2 bytes, respectively

PUSH/POP Examples

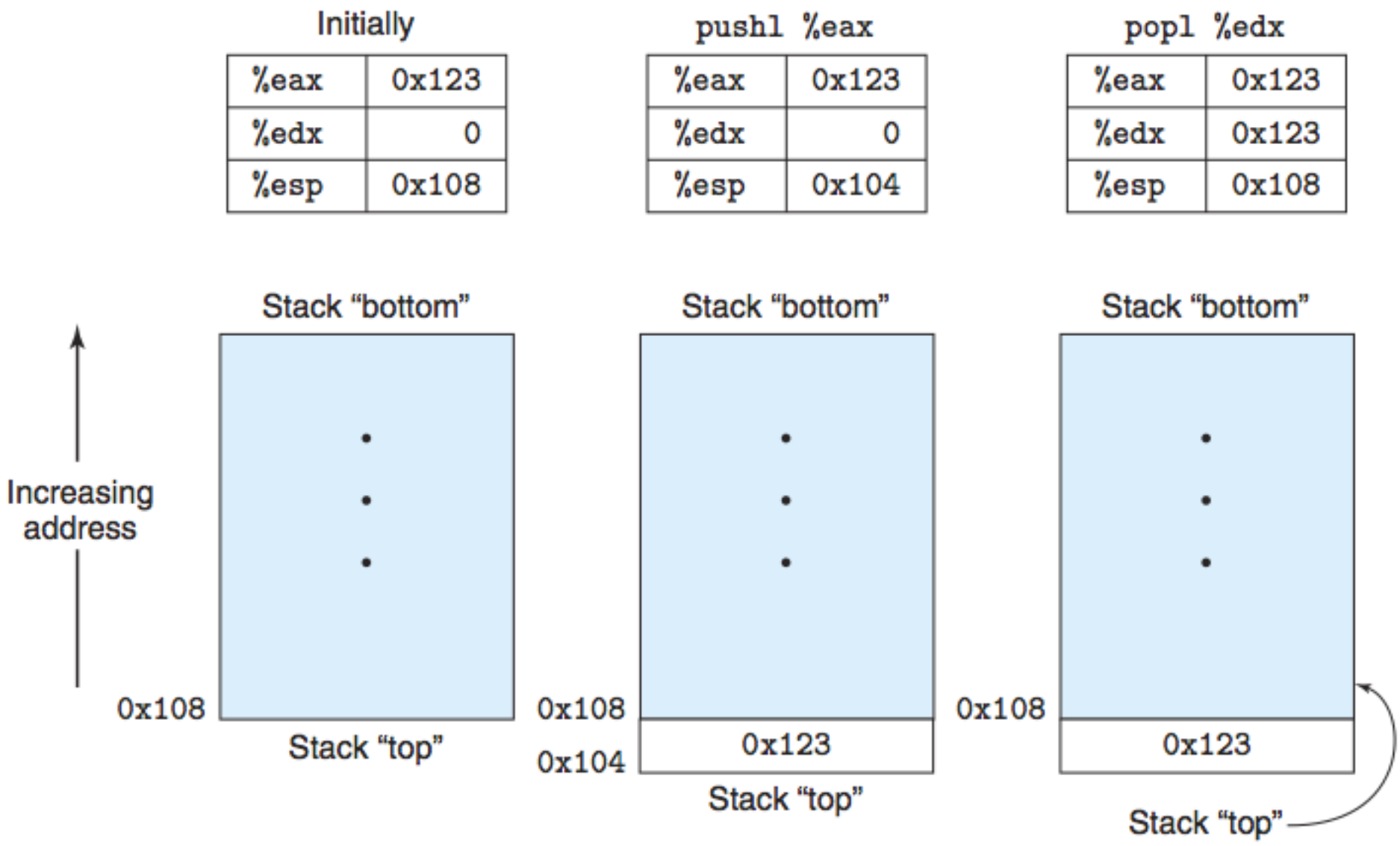
```
.section .data
x:
        .int 125

.section .text
.global test_stack
test_stack:
    pushl %ebp
    movl %esp, %ebp

    movl $24420, %ecx
    movw $350, %bx
    pushl %ecx
    pushw %bx
    pushl x
    pushl $x
    popl %eax
    popl %eax
    popw %bx
    popl %ecx

    movl %ebp,%esp
    popl %ebp
    ret
```

The PUSH and POP instructions: Example



Why is the stack so important?

- IA32 programs require a great use of the stack to:
  - Support procedure calls and return
  - Temporary data storage
  - Store local variables (next practical class)
  - Store parameters of a procedure about to call (next practical class)

- The stack is allocated in `frames` (more on this on the next pratical class)
  - State for a single procedure instantiation
  - Stack pointer `%esp` indicates stack top
  - Frame pointer `%ebp` indicates start of current frame

Procedures in Assembly

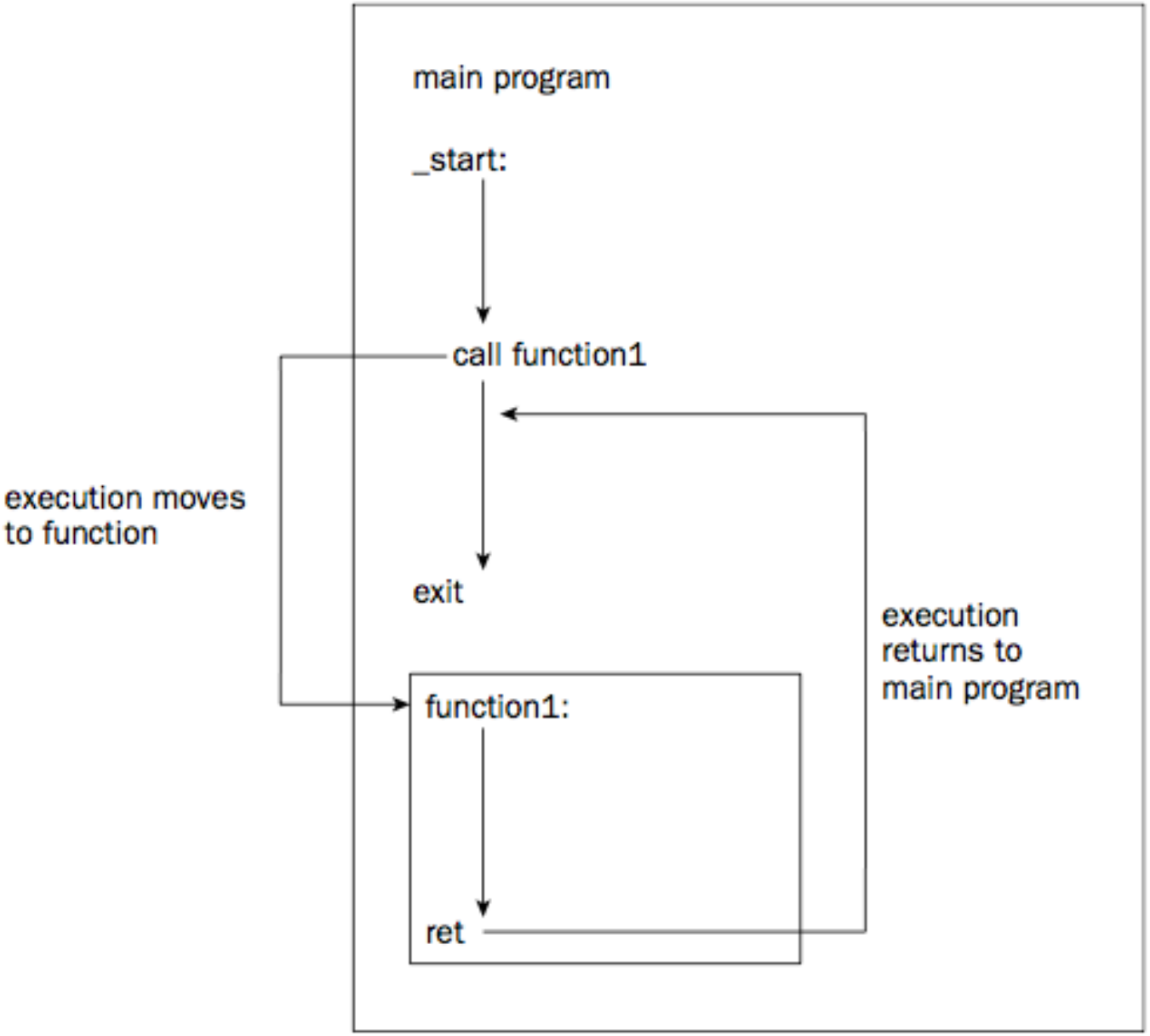
- Identified by a label
- Invoked through the `call` instruction
- End with the `ret` instruction
- The return value must be stored in the `%eax` register (or `%edx:%eax` for 64-bit values) before the execution of the `ret` instruction

Procedure Call Example

```
procedure2:
        ...
        ret

procedure1:
        ...
        call procedure2
        ...
```

Procedure calls: Execution flow



Transfer of Control

- The `call` instruction has only one parameter
  - The label that identifies the procedure to be called (the label is converted into the corresponding memory address)
- The `ret` instruction allows the execution flow to continue in the instruction immediately after the procedure call
- However, the `ret` instruction does not have any argument
- How does the `ret` instruction retrieve the correct return address?

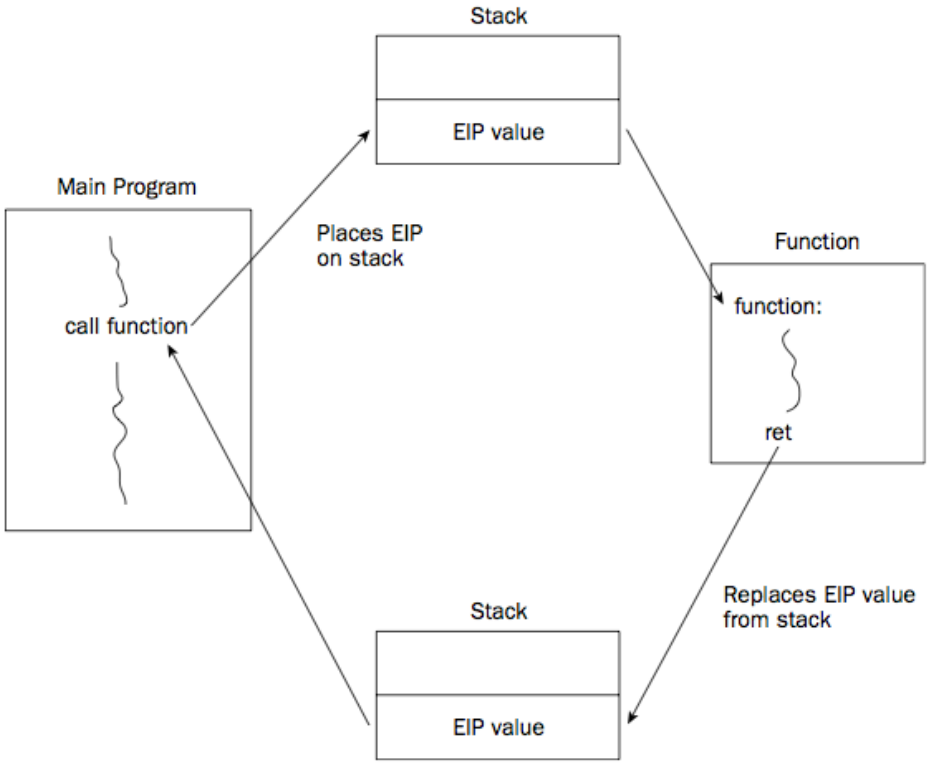
The stack and execution flow

Procedure call: **call** *label*

- Push return address on stack (address of the instruction following the call)
- Jump (*jmp*) to *label* (sets `%eip` to the corresponding address)

Procedure return: **ret**

- Pop address from stack
- Jump (*jmp*) to address
- It is up to the programmer to ensure that `%esp` is pointing to the address pushed by *call*, prior to issuing a RET instruction



Saving and restoring register state (1/2)

- The stack is the most adequate place to save the current value of a register for later restauration

Save and Restore a Register

```
...
pushl %edx          # save current value of %edx

# divide %edx:eax by %ecx
movl $0, %edx       # dividend
movl $1000, %eax    # dividend
movl $500, %ecx     # divisor
divl %ecx           # division
...                 # remainder on %edx

popl %edx           # restore old value %edx
...
```

Saving and restoring register state (2/2)

- Whenever a procedure p1 calls another procedure p2, it is possible that p2 uses some or all of the registers being used in p1

Save and Restore a Register

```
p1:
    ...
    movl $1, %ecx
    ...
    call p12          # can overwrite %ecx
    ...
    cmpl %eax, %ecx   # there is no guarantee on the current content of %ecx
    ...
```

- Therefore, there is no guarantee that those registers keep their contents after calling *p2*

- A possible (extreme) solution is to always save and restore all registers prior and after a procedure call (pushad/popad)

Register saving conventions

For efficiency sake, there is a convention that determines which registers should be saved and restored prior and after a call

"Caller Save"

- Caller saves the current contents of a set of registers prior to call
- After the procedure return, it restores those registers to their old values

"Callee Save"

- Calle saves the current contents of another set of registers before using them
- Before returning, it restores those registers to their old values

Register saving convention used in IA32/Linux

%eax, %edx, %ecx

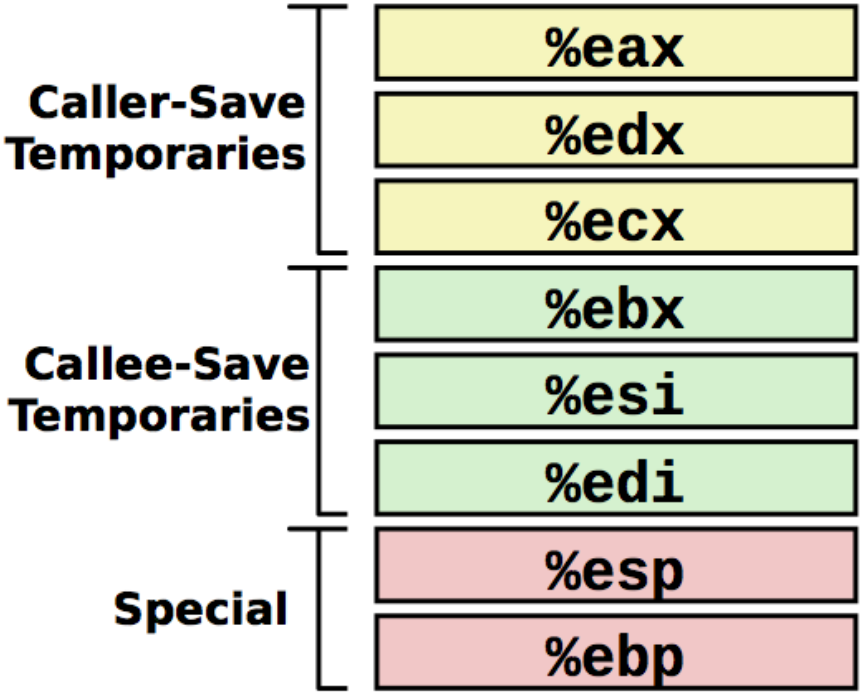
- Caller saves them on stack prior to call
- Restores them after the return of the callee procedure

%ebx, %esi, %edi

- Callee saves their current contents prior to using them
- Restores them before returning

%ebp, %esp

- Special stack registers
- Initial contents should be restored before returning



Note: It is particularly important to follow this convention every time a C function is invoked from your Assembly code

Register saving convention used in IA32/Linux: Example (1/2)

- Assume that variables a, b and c are global

Function f1

```
int f1(){
        return a + b + f2();
}
```

Function f2

```
int f2(){
        return c + 1;
}
```

- How can you implement an equivalent code in Assembly?

Register saving convention used in IA32/Linux: Example (2/2)

Function f1

```
f1:
  pushl %ebp
  movl %esp, %ebp

  # f1 is called by other functions
  pushl %ebx # save %ebx on stack

  movl a, %edx
  movl b, %ebx

  # caller is responsible for:
  # %eax, %edx and %ecx
  # save only those that are used
  pushl %edx # saves %edx on stack
  call f2
  popl %edx  # restore %edx

  addl %edx, %ebx
  addl %ebx, %eax

  popl %ebx # restore %ebx

  movl %ebp, %esp
  popl %ebp
  ret
```

Function f2

```
f2:
  pushl %ebp
  movl %esp, %ebp

  # callee is responsible for:
  # %ebx, %esi and %edi
  # save only those that are used
  pushl %ebx # save %ebx on stack

  movl c, %ebx
  incl %ebx
  movl %ebx, %eax

  popl %ebx # restore %ebx

  movl %ebp, %esp
  popl %ebp
  ret
```

Procedures in Assembly: Summary

In caller, before invoking a procedure with `call`:

- Save to stack the current contents of `%eax`, `%edx` and `%ecx`, if they are intended to be used after the return of the callee procedure with their values unchanged

In callee, before changing the contents of `%ebx`, `%esi` and `%edi`:

- Save to stack the current contents of those registers

In callee, before returning with `ret`:

- Restore the old values of `%ebx`, `%esi` and `%edi`, if used
- Move to `%eax` (or `%edx:%eax` for 64-bit values) the intended return value
- Ensure that the value at the top of the stack is the return address pushed by `call`

In caller, after the return of the callee:

- Use, if necessary, the return value of the callee procedure in `%eax` (or `%edx:%eax` for 64-bit values)
- Restore the old values of `%eax`, `%edx` and `%ecx`, if used after `call`

Practice

- Implement, in Assembly, a function `int power()` where a variable `base` is raised to a fixed `exponent`. Test it by calling your function from a C program.
- Implement another function in Assembly, `int sum_powers()`, that using the previous function, traverses an array `int nums[10]` and returns the sum of the powers of each of its elements
- Consider that the variables `base`, `exponent`, and the array `nums` are declared in C. Iteratively, populate the `base` variable with each of the elements of `nums`

Practice: C source

main.c

```c
#include <stdio.h>
#include "power.h"

int nums[]={1,2,3,4,5,6,7,8,9,10};
int base;
int exponent = 2;

int main(){

  int res,sum;

  base = 10;
  res = power();
  printf("base: %d and exponent: %d = %d\n",base,exponent,res);

  sum = sum_powers();
  printf("Sum of powers of %d = %d\n",exponent,sum);

  return 0;
}
```

Practice: Assembly source - int power()

power.S

```
power:
  pushl %ebp
  movl %esp, %ebp

  pushl %ebx              # callee saves %ebx before using it

  movl base, %ebx         # base on %ebx
  movl exponent, %ecx     # exponent on %ecx
  movl %ebx, %eax         # initial result in %eax

power_loop_start:
  cmpl $1, %ecx           # if exponenent is 1, go to end_power
  je end_power
  imull %ebx, %eax        # multiplies current result by base
  decl %ecx               # decreases the exponent
  jmp power_loop_start    # jumps to next expoenent

end_power:
  popl %ebx               # calle restores %ebx before ret

  movl %ebp, %esp
  popl %ebp
  ret
```

Practice: Assembly source - int sum_powers()

power.S

```
sum_powers:
  pushl %ebp
  movl %esp, %ebp
  pushl %esi            # callee saves %esi before using it

  movl $nums,%esi
  movl $10,%ecx         # number of elements in nums
  movl $0,%edx          # temporary counter
traverse_vec:
  movl (%esi),%eax
  movl %eax,base

  pushl %ecx            # caller saves %ecx before call
  pushl %edx            # caller saves %edx before call
  call power
  popl %edx             # caller restores %edx after power return
  popl %ecx             # caller restores %ecx after power return
  addl %eax, %edx       # sums return value of power to %edx
  addl $4, %esi         # updates address to next element in nums
  loop traverse_vec

  movl %edx,%eax
  popl %esi             # calle restores %esi before ret
  movl %ebp,%esp
  popl %ebp
  ret
```